RELOOP

Smart Recycling Marketplace

Distributed Web Application Project

Technical Document

Team

Arij Jaoua
Farah Akermi
Sarah Ben Aziza
Abderrahmen Gammoudi
Mahdi Ben Meriem
Amen Allah Ameri

ESPRIT – École d'Ingénieurs

Academic year 2026–2027

Contents

1 Introduction and Project Description **3**
1.1 Context . . . 3
1.2 Problem Statement . . . 3
1.3 Proposed Solution . . . 3
1.4 Objectives . . . 3

2 Actors **3**
2.1 Global Use Case Diagram . . . 3

3 Main Features **4**

4 Global Architecture **4**
4.1 Technology Stack . . . 5

5 Microservices Breakdown **6**
5.1 Source Code Management . . . 6

6 Data Managed by Each Service **6**
6.1 Global Class Diagram . . . 6
6.2 Domain Class Diagram . . . 7
6.3 Class Diagrams per Microservice . . . 8
6.3.1 Marketplace Service (MySQL) . . . 8
6.3.2 Collection Service (PostgreSQL) . . . 9
6.3.3 Transaction Service (MySQL) . . . 10
6.3.4 Eco Rewards Service (PostgreSQL) . . . 11
6.3.5 Environmental Impact Service (PostgreSQL) . . . 12
6.3.6 Notification and Communication Service (PostgreSQL) . . . 12
6.4 Data Dictionary per Service . . . 13
6.4.1 Marketplace Service (MySQL) . . . 13
6.4.2 Collection Service (PostgreSQL) . . . 14
6.4.3 Transaction Service (MySQL) . . . 14
6.4.4 Eco Rewards Service (PostgreSQL) . . . 14
6.4.5 Environmental Impact Service (PostgreSQL) . . . 15
6.4.6 Notification and Communication Service (PostgreSQL) . . . 15

7 API Contracts **15**

7.1 Marketplace Service 15

7.2 Collection Service 15

7.3 Transaction Service 16

7.4 Eco Rewards Service 16

7.5 Environmental Impact Service 16

7.6 Notification and Communication Service 16

8 Inter-Service Communication **16**

8.1 Synchronous Interactions 17

8.2 Asynchronous Interactions (RabbitMQ) 17

9 Security **17**

10 Conclusion **17**

1 Introduction and Project Description

1.1 Context

ReLoop is a smart recycling marketplace built as a distributed web application. It connects *citizens*, *collectors* and *recycling companies* on a single platform in order to:

- make recycling easier and more rewarding;
- promote the circular economy;
- turn waste into valuable resources.

1.2 Problem Statement

- There is a lack of connection between waste producers and recyclers.
- Recyclable materials are often thrown away.
- Citizens have difficulty finding collection services.

1.3 Proposed Solution

- A centralized digital recycling marketplace.
- Easy material listing and collection.
- Secure transactions and environmental rewards.

1.4 Objectives

ReLoop is more than a recycling marketplace: it is a complete ecosystem that connects users, collectors and recycling companies. The project aims to make recycling easier, more accessible and more rewarding, while measuring its real environmental benefits.

2 Actors

Actor	Responsibilities
Citizen	Publishes recyclable materials, browses listings, requests collections and earns points.
Collector	Manages pickup requests and verifies collected materials.
Recycling company	Purchases recyclable materials and monitors its environmental contributions.
Administrator	Manages the platform, users, rewards and system configuration.

2.1 Global Use Case Diagram

The global use case diagram (Figure 1) groups the use cases by actor. Relations marked *include* express that a use case reuses another one.

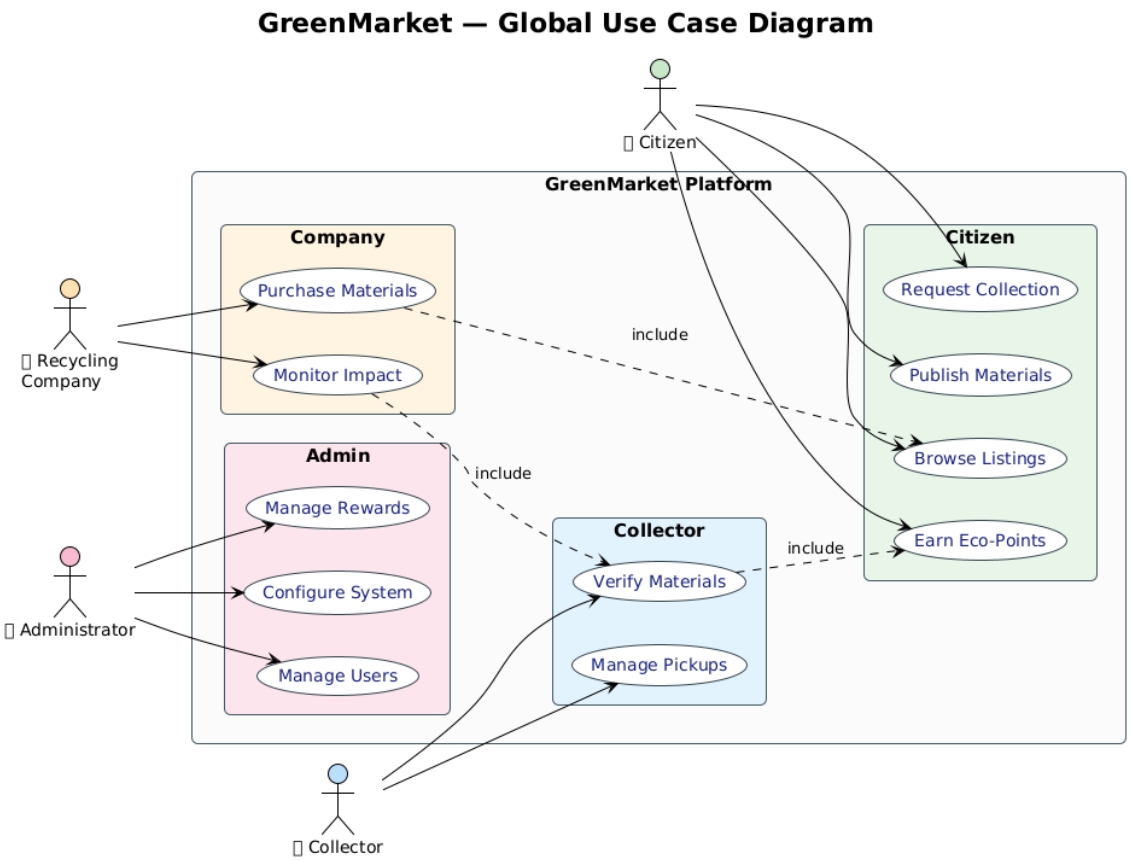


Figure 1: Global use case diagram.

3 Main Features

1. **Recycling marketplace** – publishing and discovering recyclable materials.
2. **Collection management** – scheduling and tracking pickups.
3. **Order and payment system** – managing transactions and invoices.
4. **Eco Rewards** – motivates users through points, challenges and achievements.
5. **Environmental impact** – calculates CO_2 savings and waste reduction.
6. **Notification system** – keeps users informed about important activities.
7. **Collaborative AI module** – waste recognition and valuation assistance.

4 Global Architecture

ReLoop follows a **microservices architecture** organised in five layers (Figure 2); asynchronous flows are detailed in Section 8.

#	Layer	Content
1	Client layer	Web / mobile client communicating over HTTPS.
2	Security & entry layer	API Gateway (Spring Security, OAuth2, JWT) acting as the single entry point; Keycloak as identity provider validating JWT tokens.
3	Business microservices	Marketplace, Collection, Transaction, Eco Rewards, Environmental Impact, Notification.
4	Data layer	One database per service (low coupling, data independence).
5	Messaging layer	RabbitMQ event bus for asynchronous communication.

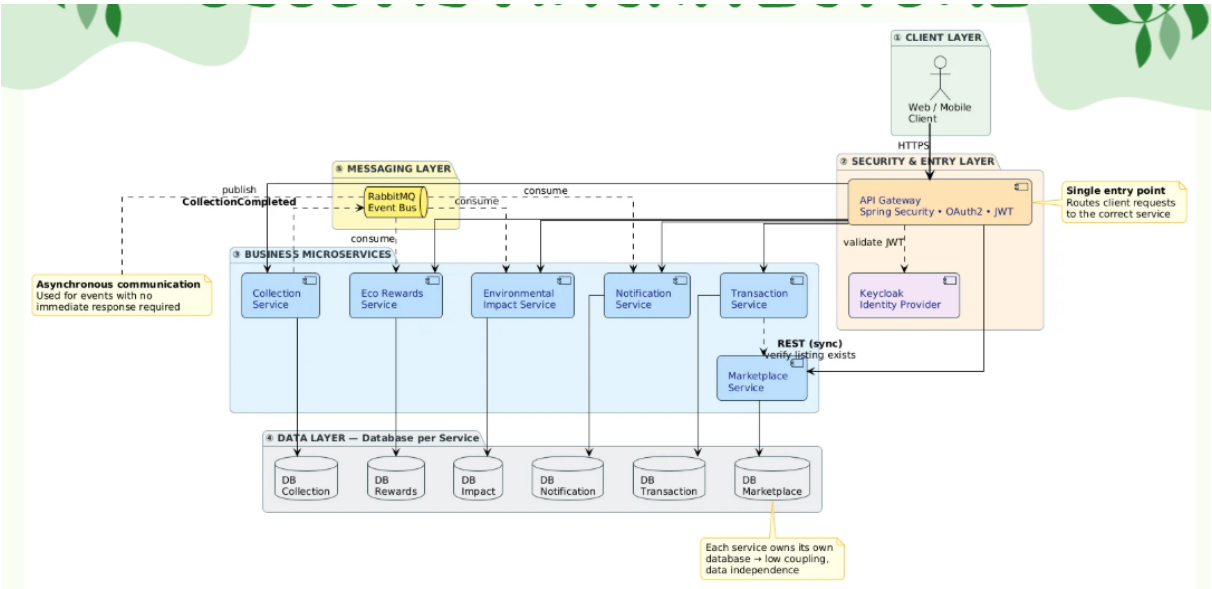


Figure 2: Global architecture: client, security, business microservices, data and messaging layers.

4.1 Technology Stack

Category	Technology	Role
Backend	Spring Boot	Microservices development framework
Databases	PostgreSQL, MySQL	Relational storage, one database per service
Communication	RabbitMQ	Asynchronous messaging (events)
	OpenFeign	Synchronous service-to-service calls
Security	Keycloak, Spring Security, OAuth2, JWT	Identity and access management, authentication and authorization
Development tools	Git, GitHub, IntelliJ IDEA	Version control, source hosting, development environment

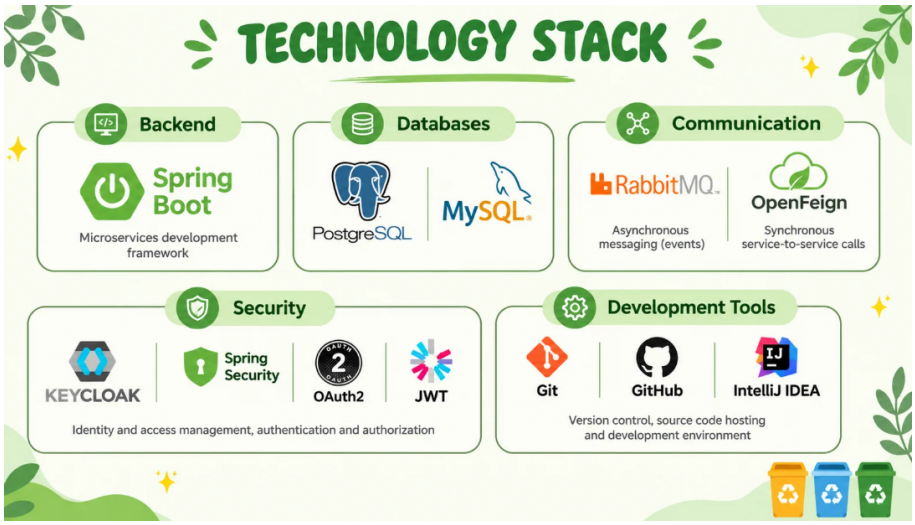


Figure 3: Technology stack.

5 Microservices Breakdown

Each microservice is assigned to one team member and owns its own database.

Student	Microservice	Database	Responsibility
1	Marketplace	MySQL	Publishing and browsing recyclable materials
2	Collection	PostgreSQL	Scheduling and tracking pickups
3	Transaction	MySQL	Payments and invoices
4	Eco Rewards	PostgreSQL	Wallets, points and rewards
5	Environmental Impact	PostgreSQL	Sustainability reports (CO_2, waste, water)
6	Notification & Communication	PostgreSQL	User notifications triggered by business events

5.1 Source Code Management

The source code is hosted in a public GitHub repository (`Reloop-4DAS-I2-Reloopers`) with a `Backend` folder (one module per service) and a `Documentation` folder.

6 Data Managed by Each Service

6.1 Global Class Diagram

The global domain model is shown in Figure 4.

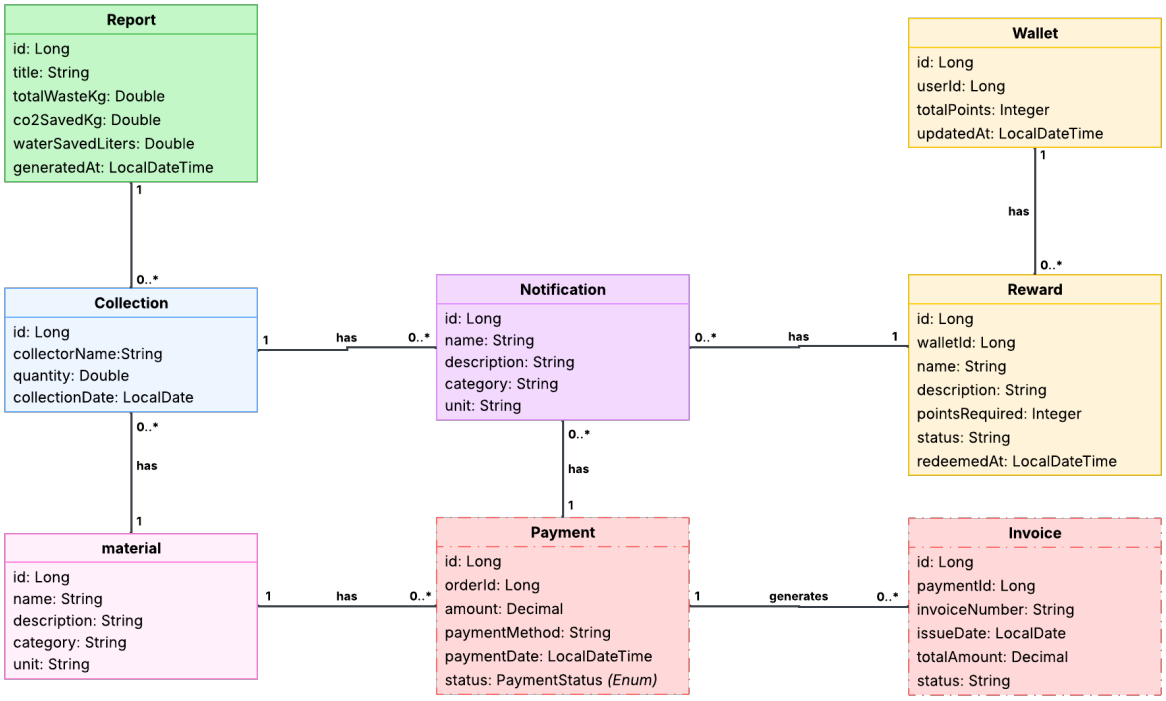


Figure 4: Global class diagram.

6.2 Domain Class Diagram

The domain class diagram groups the classes by microservice and shows how the services communicate: dashed arrows represent asynchronous events (RabbitMQ) or synchronous calls (REST).

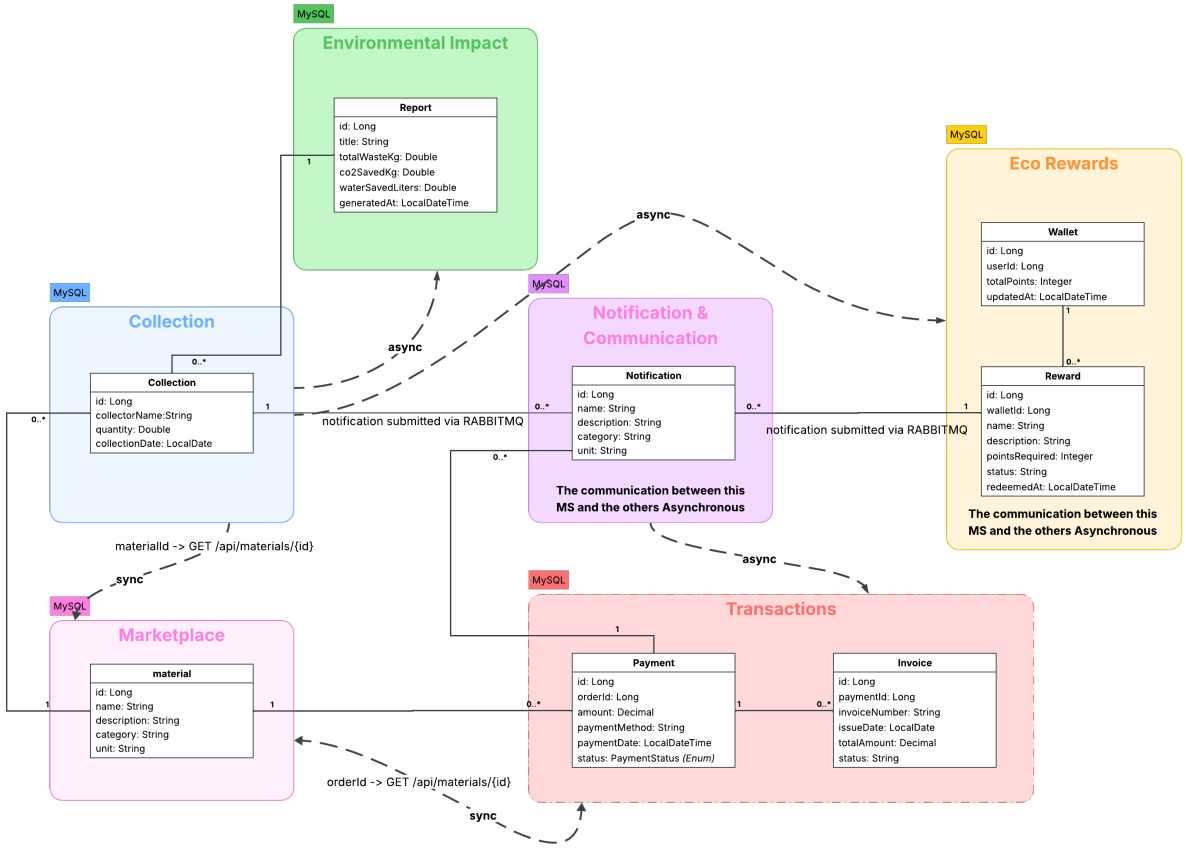


Figure 5: Domain class diagram by microservice.

6.3 Class Diagrams per Microservice

Each microservice has its own domain classes, attributes and business operations. The following diagrams present the internal class structure of each service, highlighting the main entities and their responsibilities.

6.3.1 Marketplace Service (MySQL)

The Marketplace Service manages recyclable materials available on the platform. The `Material` class represents the information associated with each recyclable material, including its name, description, category and unit.

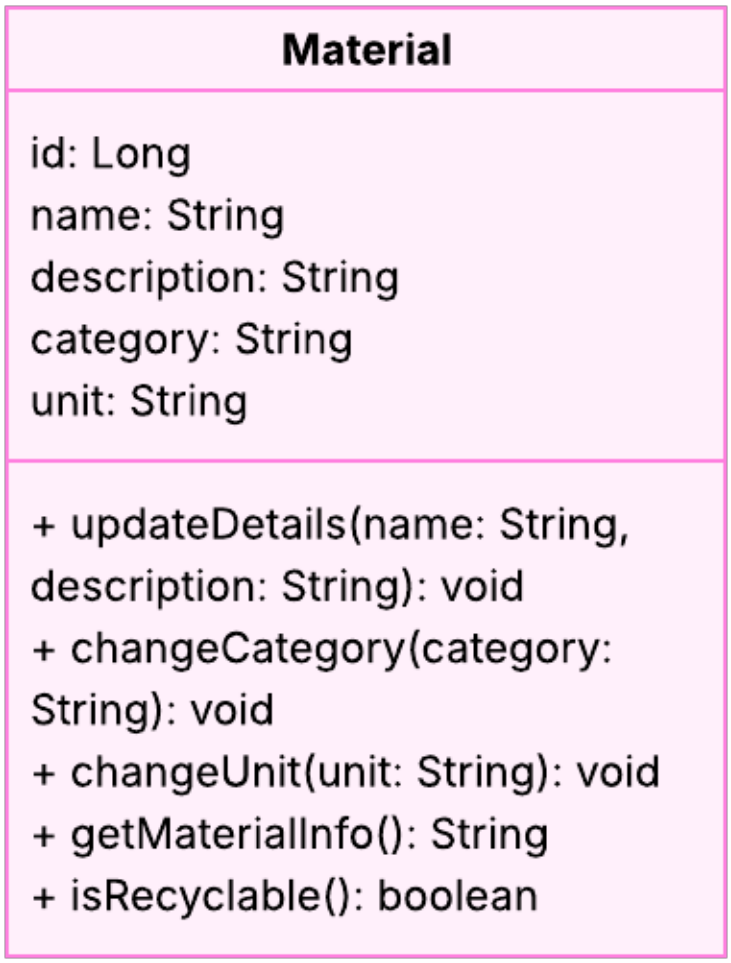


Figure 6: Marketplace Service – Material Class Diagram.

6.3.2 Collection Service (PostgreSQL)

The Collection Service handles collection scheduling, collector assignment and quantity management. The `Collection` class represents a collection operation and provides methods for scheduling, updating, completing, cancelling and verifying collections.

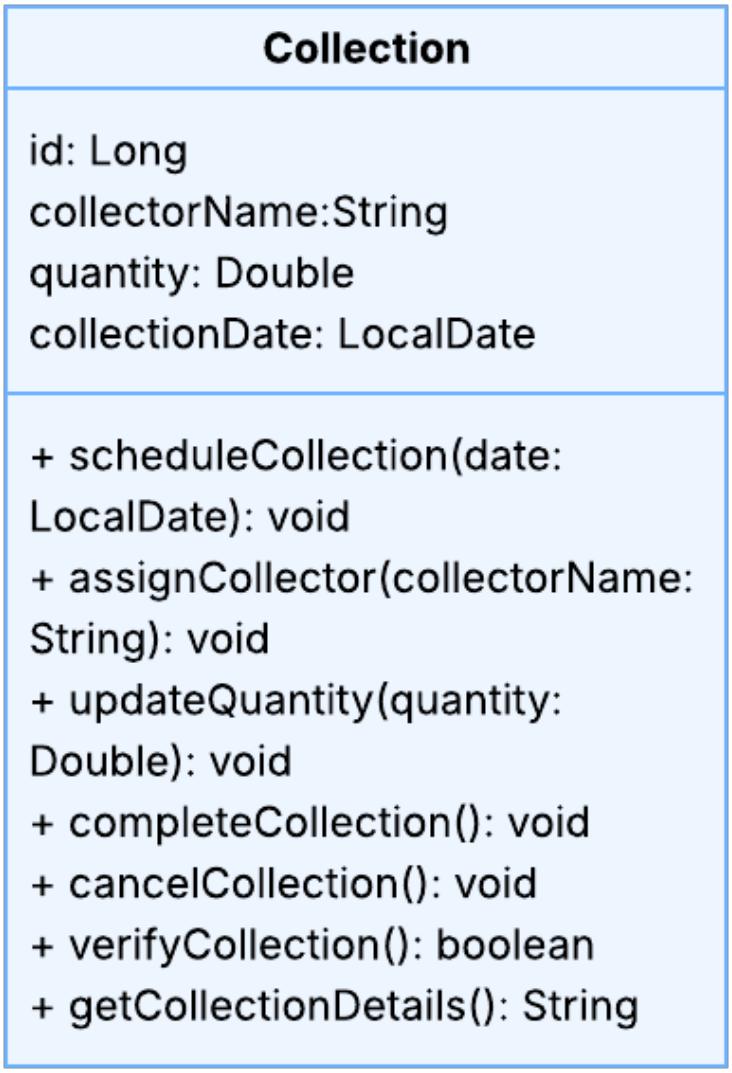


Figure 7: Collection Service – Collection Class Diagram.

6.3.3 Transaction Service (MySQL)

The Transaction Service manages financial operations related to platform orders. It includes two main classes:

- **Payment:** manages payment processing, validation, confirmation, failure and refunds.
- **Invoice:** manages invoice generation, total calculation, status updates and downloading.

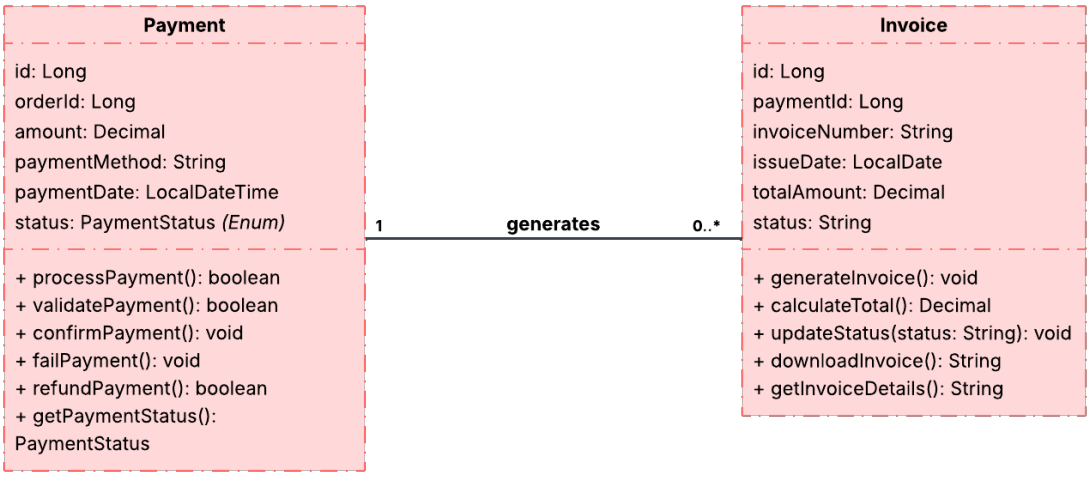


Figure 8: Transaction Service – Payment and Invoice Class Diagram.

6.3.4 Eco Rewards Service (PostgreSQL)

The Eco Rewards Service manages users' reward points and reward redemption. It includes two main classes:

- **Wallet:** stores the user's total points and manages the points balance.
- **Reward:** represents available rewards, required points and redemption status.

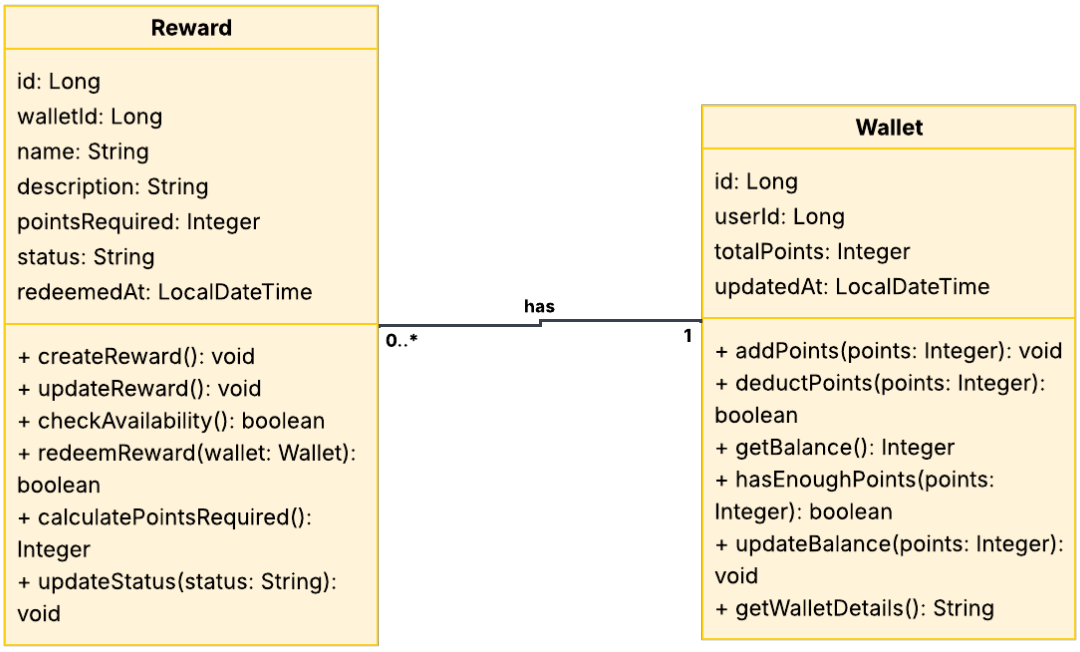


Figure 9: Eco Rewards Service – Wallet and Reward Class Diagram.

6.3.5 Environmental Impact Service (PostgreSQL)

The Environmental Impact Service measures the environmental benefits generated through recycling activities. The `Report` class stores environmental indicators, including total recycled waste, CO_2 savings and water savings.

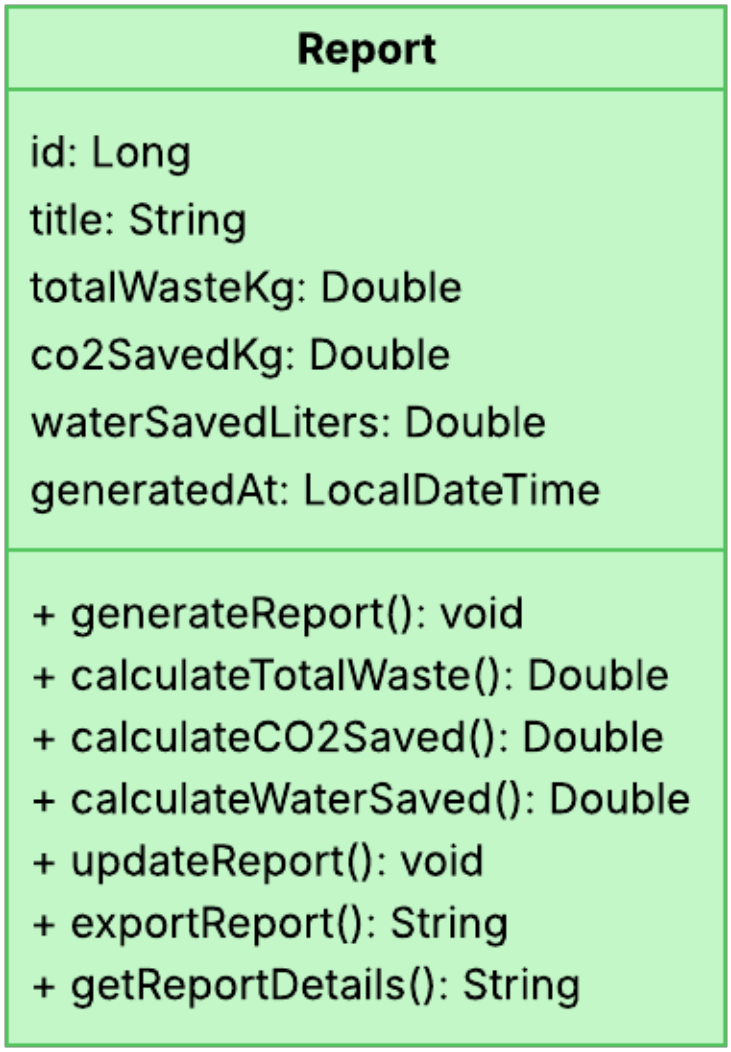


Figure 10: Environmental Impact Service – Report Class Diagram.

6.3.6 Notification and Communication Service (PostgreSQL)

The Notification Service manages notifications generated by platform activities. The `Notification` class provides operations for creating, sending, reading, deleting and updating notification messages.

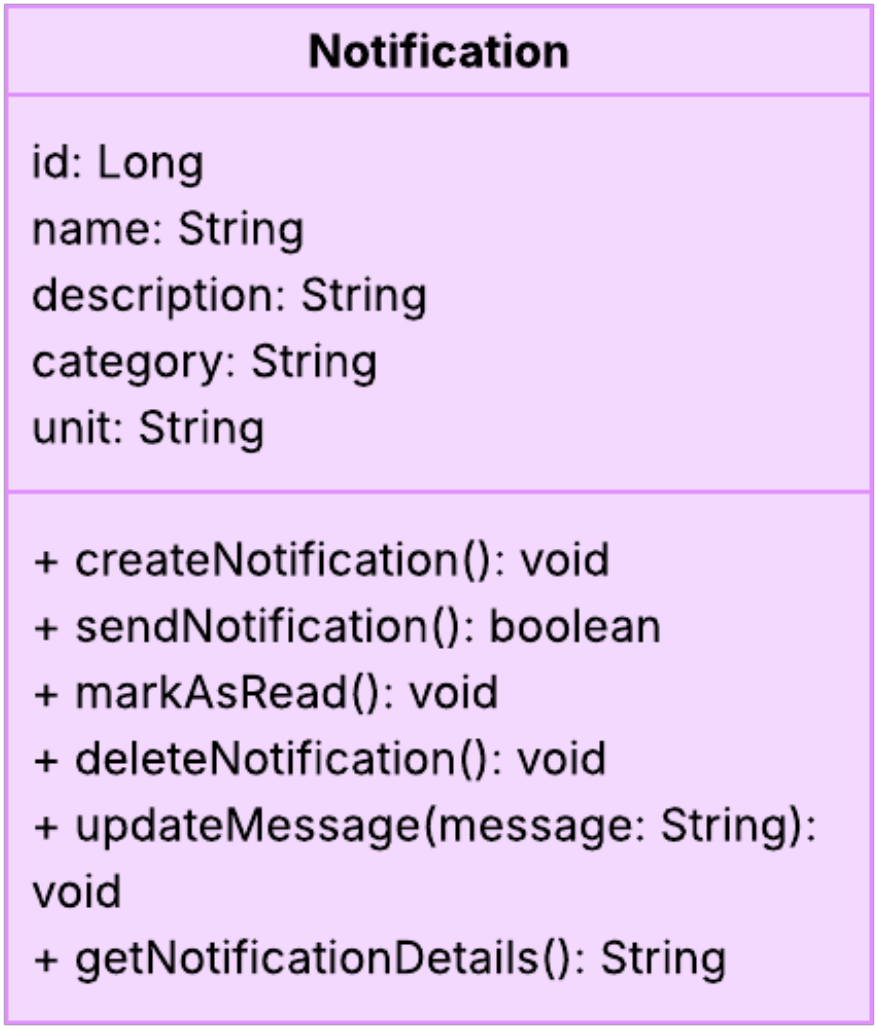


Figure 11: Notification and Communication Service – Notification Class Diagram.

6.4 Data Dictionary per Service

This section describes the main entities and attributes managed by each microservice.

6.4.1 Marketplace Service (MySQL)

Entity	Attribute	Type / Description
Material	id	Long – identifier
	name	String
	description	String
	category	String
	unit	String

6.4.2 Collection Service (PostgreSQL)

Entity	Attribute	Type / Description
Collection	id	Long – identifier
	collectorName	String
	quantity	Double
	collectionDate	LocalDate

6.4.3 Transaction Service (MySQL)

Entity	Attribute	Type / Description
Payment	id	Long
	orderId	Long
	amount	Decimal
	paymentMethod	String
	paymentDate	LocalDateTime
	status	PaymentStatus (enum)
Invoice	id	Long
	paymentId	Long – reference to Payment
	invoiceNumber	String
	issueDate	LocalDate
	totalAmount	Decimal
	status	String

6.4.4 Eco Rewards Service (PostgreSQL)

Entity	Attribute	Type / Description
Wallet	id	Long
	userId	Long
	totalPoints	Integer
	updatedAt	LocalDateTime
Reward	id	Long
	walletId	Long – reference to Wallet
	name	String
	description	String
	pointsRequired	Integer
	status	String
	redeemedAt	LocalDateTime

6.4.5 Environmental Impact Service (PostgreSQL)

Entity	Attribute	Type / Description
Report	id	Long
	title	String
	totalWasteKg	Double
	co2SavedKg	Double
	waterSavedLiters	Double
	generatedAt	LocalDateTime

6.4.6 Notification and Communication Service (PostgreSQL)

Entity	Attribute	Type / Description
Notification	id	Long
	name	String
	description	String
	category	String
	unit	String

7 API Contracts

All endpoints are exposed through the API Gateway.

7.1 Marketplace Service

Method	Endpoint	Description	Codes
GET	`/api/materials`	Retrieve all materials	200, 500
GET	`/api/materials/{id}`	Retrieve material details	200, 404
POST	`/api/materials`	Create a material	201, 400
PUT	`/api/materials/{id}`	Update a material	200, 400, 404

7.2 Collection Service

Method	Endpoint	Description	Codes
GET	`/api/collections`	Retrieve all collections	200, 500
GET	`/api/collections/{id}`	Retrieve collection details	200, 404
POST	`/api/collections`	Create a collection request	201, 400
PUT	`/api/collections/{id}`	Update collection information	200, 400, 404

7.3 Transaction Service

Method	Endpoint	Description	Codes
POST	`/api/payments`	Process a payment	201, 400, 409
GET	`/api/payments/{id}`	Retrieve payment details	200, 404
GET	`/api/invoices/{id}`	Retrieve an invoice	200, 404

7.4 Eco Rewards Service

Method	Endpoint	Description	Codes
GET	`/api/rewards/wallet/{userId}`	Retrieve user wallet	200, 404
GET	`/api/rewards`	Retrieve available rewards	200, 500
POST	`/api/rewards/redeem`	Redeem a reward	200, 400, 409
GET	`/api/rewards/history/{userId}`	Retrieve points history	200, 404

7.5 Environmental Impact Service

Method	Endpoint	Description	Codes
GET	`/api/reports`	Retrieve sustainability reports	200, 500
GET	`/api/reports/{id}`	Retrieve record details	200, 404
POST	`/api/reports/generate`	Generate a sustainability report	201, 400

7.6 Notification and Communication Service

Method	Endpoint	Description	Codes
POST	`/api/notifications/send`	Send a notification	202, 400
GET	`/api/notifications/{id}`	Retrieve notification details	200, 404
GET	`/api/notifications/user/{userId}`	Retrieve user notifications	200, 404

8 Inter-Service Communication

Two communication styles are used:

- **Synchronous (REST via OpenFeign)** when an immediate answer is required.
- **Asynchronous (RabbitMQ events)** for events with no immediate response required, which keeps services loosely coupled.

8.1 Synchronous Interactions

Interaction	Protocol	Purpose
Marketplace ↔ Transaction	REST	Check listing details and availability (verify listing exists) before creating an order.
Collection → Marketplace	REST	Retrieve listing information.

8.2 Asynchronous Interactions (RabbitMQ)

Event	Publisher	Consumers / Purpose
`CollectionCompleted`	Collection	Eco Rewards (award points), Environmental Impact (update statistics), Notification.
`PaymentCompleted`	Transaction	Notification.
`RewardRedeemed`	Eco Rewards	Notification.
`ImpactReportGenerated`	Environmental Impact	Notification.

The Notification Service consumes the relevant business events published by the other services in order to trigger user notifications.

9 Security

- All client traffic goes through the **API Gateway**, the single entry point that routes requests to the correct service.
- Authentication and authorization rely on **OAuth2** and **JWT** tokens issued by **Keycloak** (identity provider).
- The gateway validates the JWT with Keycloak before forwarding the request; Spring Security enforces access rules.
- Access is role-based, matching the four actors: citizen, collector, recycling company and administrator.

10 Conclusion

ReLoop is a complete recycling ecosystem built on six independent Spring Boot microservices, each with its own database, secured through an API Gateway with Keycloak, and coupled through REST calls and RabbitMQ events. The design makes recycling easier, more accessible and more rewarding while measuring its real environmental benefits.